

BASE DE DATOS II – *TRABAJO PRÁCTICO*
INTEGRADOR - ETAPA 2

Grupo: 144

Integrantes: Damian Tristant | Ximena Maribel Sosa

Dominio Elegido: Gestión de Órdenes de Trabajo - Taller de Carpintería

Bloque 1: Operaciones CRUD desde la Aplicación

Tecnología utilizada

Se desarrolló un script backend en Node.js utilizando el Driver Nativo de MongoDB (mongodb v6), tal como se trabajó en la Unidad 7 de la materia. Esta elección permite conectarse directamente al clúster de MongoDB Atlas sin capas intermedias, haciendo explícito el flujo de comunicación entre la aplicación y el motor de base de datos.

Conexión a MongoDB Atlas

La conexión se establece mediante un objeto MongoClient inicializado con el string de conexión del clúster. Al ejecutar client.connect(), el Driver abre un pool de conexiones TCP hacia los nodos del Replica Set en Atlas. Una vez finalizadas todas las operaciones, client.close() libera esas conexiones.

```
PS D:\TUP\2 año\Bases de Datos II\TPI Parte 2 - Grupo 144> node crud.js
Conexión exitosa a MongoDB Atlas

--- CREATE: Insertar nuevo cliente ---
Documento insertado con _id: new ObjectId('6a3c75632501d77691967271')

--- READ: Listar clientes activos ---
Se encontraron 3 cliente(s) activo(s):
- Carlos Rodríguez | carlos.rodriguez@email.com
- María González | maria.gonzalez@email.com
- Roberto Sánchez | roberto.sanchez@email.com

--- UPDATE: Actualizar email del cliente ---
Documentos modificados: 1

--- DELETE LÓGICO: Desactivar cliente ---
Documentos desactivados: 1
- Roberto Sánchez | activo: false

Ciclo CRUD completado correctamente.
Conexión cerrada.
```

CREATE – Inserción de un nuevo cliente

Se utiliza el método insertOne() para registrar un nuevo documento en la colección clientes. El documento incluye el campo activo: true desde su creación, siguiendo la convención de Baja Lógica definida en la Parte 1.

READ – Consulta de clientes activos

Se utiliza find() con el filtro { activo: true } para listar únicamente los clientes operativos. Este filtro es la implementación directa de la Baja Lógica en el código: los documentos marcados con activo: false existen físicamente en la base de datos pero no son devueltos por la consulta, comportándose como si hubieran sido eliminados desde la perspectiva del usuario.

UPDATE – Modificación de un campo específico

Se utiliza `updateOne()` combinado con el operador `$set` para modificar únicamente el campo indicado, sin alterar el resto del documento. Esto es importante porque en MongoDB un `update` sin `$set` reemplazaría el documento completo.

DELETE Lógico – Desactivación de un registro

En lugar de ejecutar `deleteOne()` que eliminaría el documento físicamente, se realiza un `updateOne()` que cambia el campo `activo` a `false`. El documento permanece en la base de datos para auditoría e historial, pero queda excluido de todas las consultas operativas que filtran por `activo: true`.

Bloque 2: Mecanismo de Backup y Resguardo

Script de Backup (backup.bat)

Se desarrolló un script ejecutable para Windows que automatiza el respaldo físico de la base de datos. Al ejecutarse realiza las siguientes acciones:

1. Obtiene la fecha actual del sistema operativo en formato YYYY-MM-DD
2. Crea la estructura de carpetas `resguardos_tpi\2026-06-24\` usando rutas relativas
3. Ejecuta `mongodump` conectándose remotamente al clúster de Atlas y descarga un snapshot completo de la base de datos

El resultado es una carpeta con archivos `.bson` (documentos de cada colección en formato binario) y archivos `.metadata.json` (estructura, índices y validadores de cada colección).

```
Carpeta creada: resguardos_tpi\2026-06-24
Iniciando backup...
2026-06-24T21:59:19.427-0300    writing 'tpi_taller_carpinteria.clientes' to 'resguardos_tpi\2026-06-24\tpi_taller_carpinteria\clientes.bson'
2026-06-24T21:59:19.430-0300    writing 'tpi_taller_carpinteria.ordenes_trabajo' to 'resguardos_tpi\2026-06-24\tpi_taller_carpinteria\ordenes_trabajo.bson'
2026-06-24T21:59:19.431-0300    writing 'tpi_taller_carpinteria.productos' to 'resguardos_tpi\2026-06-24\tpi_taller_carpinteria\productos.bson'
2026-06-24T21:59:19.431-0300    writing 'tpi_taller_carpinteria.materiales' to 'resguardos_tpi\2026-06-24\tpi_taller_carpinteria\materiales.bson'
2026-06-24T21:59:19.683-0300    done dumping 'tpi_taller_carpinteria.clientes' (5 documents)
2026-06-24T21:59:19.732-0300    done dumping 'tpi_taller_carpinteria.ordenes_trabajo' (4 documents)
2026-06-24T21:59:19.781-0300    done dumping 'tpi_taller_carpinteria.productos' (4 documents)
2026-06-24T21:59:19.821-0300    done dumping 'tpi_taller_carpinteria.materiales' (3 documents)
Backup completado en: resguardos_tpi\2026-06-24
Presione una tecla para continuar . . . |
```

Parte 2 - Grupo 144\resguardos_tpi\2026-06-24\tpi_taller_carpinteria			Buscar en tpi_
C:\Users\NoxiePC\AppData\Local\Microsoft\Edge\User Data\Default\Index			
C:\Windows\System32\DriverStore\FileRepository			
			Tamaño
clientes.bson	24/6/2026 21:59	Archivo BSON	1 KB
clientes.metadata	24/6/2026 21:59	Archivo de origen ...	1 KB
materiales.bson	24/6/2026 21:59	Archivo BSON	1 KB
materiales.metadata	24/6/2026 21:59	Archivo de origen ...	1 KB
ordenes_trabajo.bson	24/6/2026 21:59	Archivo BSON	3 KB
ordenes_trabajo.metadata	24/6/2026 21:59	Archivo de origen ...	1 KB
prelude	24/6/2026 21:59	Archivo de origen ...	1 KB
productos.bson	24/6/2026 21:59	Archivo BSON	2 KB
productos.metadata	24/6/2026 21:59	Archivo de origen ...	1 KB

Análisis RTO y RPO

Recovery Point Objective (RPO)

Define la cantidad máxima de datos que el negocio puede permitirse perder. Para "Muebles del Bosque" el RPO es de 24 horas. El taller opera en horario comercial y mantiene registros físicos en papel como respaldo paralelo. Perder menos de un día de datos digitales es aceptable porque puede reconstruirse a partir de los comprobantes físicos.

Recovery Time Objective (RTO)

Define el tiempo máximo de inactividad tolerable antes de que el negocio se vea seriamente afectado. Para este dominio el RTO es de 24 horas. Al no ser un sistema de disponibilidad continua como un e-commerce o una entidad bancaria, una interrupción nocturna o de fin de semana es tolerable siempre que el sistema sea restaurado antes del siguiente turno operativo.

Estrategia elegida: Backup Completo diario

Se implementó un backup completo (Full Backup) ejecutado una vez por día. Esta estrategia es la más simple de restaurar: ante una falla, basta con ejecutar mongorestore apuntando a la carpeta del día anterior para recuperar el estado completo de la base de datos, sin necesidad de aplicar backups incrementales encadenados.

Conclusión: Desafíos de la Comunicación Cliente-Servidor

Conectar una aplicación Node.js a MongoDB Atlas implicó comprender el flujo real de comunicación entre el cliente y el servidor. El principal aprendizaje fue entender que la conexión no es instantánea ni permanente: el Driver establece un pool de conexiones TCP al llamar a `client.connect()`, ejecuta las operaciones, y las libera con `client.close()`. Omitir el cierre deja la aplicación colgada esperando liberar recursos.

Trabajar con un clúster en la nube (Atlas) en lugar de una instancia local introdujo la latencia de red como factor real: cada operación viaja hasta los servidores de AWS en São Paulo, lo que hace visible algo que en local es imperceptible.

Finalmente, implementar la baja lógica en código reforzó su valor práctico: el filtro `{ activo: true }` en el `READ` es lo que hace que toda la arquitectura de baja lógica diseñada en la Parte 1 tenga efecto real en la aplicación. Sin ese filtro, un usuario vería datos "eliminados" sin saberlo. Por último, cabe mencionar que en un entorno productivo real las credenciales de conexión no se incluirían directamente en el código fuente sino en variables de entorno (archivo `.env`), minimizando el riesgo de exposición ante accesos no autorizados al repositorio.